계좌이체·잔고관리
금융 서비스 관점 코드 리뷰

ACCOUNT SERVICE

금액의 정확성, 거래의 단일 처리, 보안 경계와 운영 복구를 기준으로 검토한 실무 리뷰 보고서입니다.

> 검토 결론: 실자금 운영 승인은 보류해야 합니다. 기본 거래 구조는 갖추었지만 멱등성 원자성, 금액 정밀도, 민감정보 로그 및 배포 통제에 우선 보완이 필요합니다.

검토 범위	확인 결과
애플리케이션 Java	51개 파일 / 1,928줄
테스트 Java	24개 파일 / 1,912줄
Git 추적 파일	105개 + 로컬 설정 1개 구조 확인
중요도	P1 8건 / P2 11건 / 확정 P0 0건
실행 검증	기존 전체 73개 통과 / 추가 감사 검증 10개 통과

기준 커밋 7d6f766f7722de5b7ec84e88d4a22537e882ffb2

대상 C:/Users/정유현/Desktop/project/account

Docker 재검증 반영판 · 원본 소스 수정 없음 · 운영 환경 접속 없음

이 문서는 코드와 배포 산출물에 대한 엔지니어링 검토입니다. 금융 규제 준수 인증이나 실제 운영 시스템의 보안 감사 결과를 의미하지 않습니다.

먼저 읽을 판단과 수정 순서

단순히 @Transactional, JWT, Redis, Kubernetes가 존재하는지만으로 금융 거래의 안전성을 판단할 수 없습니다. 실패가 발생하는 경계에서도 "돈이 한 번만, 정확한 금액으로, 허가된 주체에 의해 움직이는가"가 핵심입니다.

우선순위	판단	대표 항목
P1 / 출시 차단	실자금 처리 전 해결. 조건부 항목은 해당 조건을 배포 증거로 해소해야 함.	R01-R08
P2 / 운영 전 보완	복구·감사·릴리스·조회 정확성을 위해 계획된 수정 필요.	R09-R19
P0 / 긴급 사고	이번 검토에서 실제 운영 사고 또는 모든 조건에서의 즉시 악용을 확인한 항목은 없음.	0건

가장 먼저 고칠 세 가지

1. R01-R02: 멱등성 저장과 거래 커밋을 원자적으로 묶고 사용자·요청 내용별로 재생 범위를 제한합니다.

2. R03: 허용 통화 단위와 금액 정밀도를 확정하고, DB에 도달하기 전에 과도한 자릿수와 한도 초과를 차단합니다.

3. R05-R08: 비밀번호 로그, 로그인 제한 경계, Redis 배포 누락, 운영 자동 DDL을 정리합니다.

현재 구조에서 유지할 부분

조건부 원자적 출금 UPDATE, 일관된 두 계좌 갱신 순서, 잔액·거래 이력의 동일 서비스 트랜잭션, 읽기 전용 조회, 이력 fetch join, 비밀번호 해싱, 소유자 검사, MySQL/Redis Testcontainers 사용은 유지할 가치가 있습니다.

읽는 순서

요약과 범위 → 전체 요청 흐름 → 지적 사항 목록 → R01-R19 상세 → 개선 설계와 검증 계획 → 파일별 검토 목록. 각 상세 항목에는 근거 수준, 위치, 발생 조건, 최소 수정안, 완료 기준을 함께 적었습니다.

검토 범위와 검증의 한계

구분	이번에 수행한 내용
전체 정적 검토	Java 75개 전체, 애플리케이션/테스트 설정, Dockerfile, Gradle, CI/CD, k8s 6개, 설계 문서와 README를 검토.
기존 테스트 실행	Docker 28.3.2 연결 후 기존 22개 테스트 클래스의 73개 전부 통과. MySQL/Redis 통합·동시성·배치 및 H2 경로 포함. Gradle 9.4.1 / Java 17 toolchain / 오프라인 빌드.
추가 감사 검증	격리 복사본에서 단위 재현 E1-E5 5개와 실제 MySQL 기반 D1-D5 5개 통과. 현재 위험 동작의 재현과 정상 동시 요청 대조이며 수정 완료를 의미하지 않음.
실행 범위의 한계	테스트 컨테이너와 MockMvc 기반 로컬 검증. 장시간 부하, 프로세스 강제 종료, 실제 네트워크 응답 유실, 운영 DB 복원 실험은 수행하지 않음.
외부 운영 미확인	실제 DB schema/권한, 클러스터 상태, TLS 종료 장비, 인그레스 신뢰 프록시, 브랜치 보호, 배포 이력, 백업/복원은 확인하지 않음.

근거 표시 방식

E1-E5는 프로젝트 클래스를 사용한 단위 재현, D1-D5는 실제 MySQL을 사용한 통합 검증입니다. "정적 확인"은 코드·설정에서 확인한 사실이며, "조건부"는 배포 환경이나 업무 계약의 추가 확인이 필요한 판단입니다.

검토 대상에서 제외한 항목

build/.gradle/.git/.idea/.omc의 생성물과 내부 캐시, 개인 도구 설정은 제품 소스 범위에서 제외했습니다. Gradle wrapper JAR는 존재·파일 해시 수준으로만 확인했으며 바이너리를 역공학하지 않았습니다. 외부 라이브러리 전체 소스 및 CVE/SBOM 검사는 수행하지 않았습니다.

application-local.yaml은 값의 노출 없이 구조만 확인했습니다. 문서에 DB 비밀번호·JWT 시크릿 값은 포함하지 않았습니다. 검토 기준은 시작 시점의 파일 복사본이며 소스 변경 여부는 마지막에 다시 대조합니다.

현재 요청 흐름과 지켜야 할 불변식

클라이언트 → CorrelationId / RateLimit 필터
추적 ID 부여 · IP 기반 Redis 토큰 버킷 · Redis 오류 시 허용

↓

Spring Security → TransactionController
JWT 검증 · 로그인 주체 추출 · 요청 DTO 검증

↓

IdempotencyService
키 저장 → 기존 응답 재생 또는 action 호출 → 완료 응답 저장

↓

TransactionService → MySQL
계좌 소유 검사 → 조건부 UPDATE → 거래 이력 INSERT → 커밋

↓

별도 운영 경로: 배치 / 관측 / 배포
전체 잔액 대사 · 관리 포트 9090 · GitHub Actions와 k3s

불변식	현재 판단
동일 요청은 한 번만 반영	거래와 키 완료의 분리 때문에 R01-R02에서 깨질 수 있음.
이체 전후 금액 총합 보존	과도한 소수 정밀도 요청으로 총액 증가를 실제 MySQL에서 확인. R03 / D1.
고객이 임의로 돈을 만들 수 없음	모의 입금과 실입금의 업무 경계가 필요함. R04.
누가 무엇을 확정했는지 추적	소유자 검사는 있으나 커밋 확정 감사 연결은 보완 필요. R10.
이상 탐지와 복구가 가능	대사 배치는 존재. 마감 의미·결과 저장·복구는 R11-R13.

주의: 현재 주 거래 경로는 @Version을 비교하는 낙관적 갱신이 아니라 DB의 조건부 원자적 UPDATE입니다. version을 증가시키지만 WHERE 조건에서 이전 version을 비교하지 않습니다. 과거 README와 일부 주석은 이 변경을 반영하지 못했습니다.

전체 지적 사항

ID	등급	핵심 문제	상세
R01	P1	거래 커밋과 멱등성 완료 저장이 분리되어 중복 처리 가능	6쪽
R02	P1	멱등성 키에 사용자·거래 종류·요청 내용이 결합되지 않음	7쪽
R03	P1	금액 scale·상한 미검증으로 이체 시 총액 보존 위반	8쪽
R04	P1	일반 사용자 입금에 자금 유입 증빙·권한 경계가 없음	9쪽
R05	P1	검증 예외 원문 로그에 비밀번호 등 거절된 값이 노출	10쪽
R06	P1	X-Forwarded-For를 무조건 신뢰하여 요청 제한 우회 가능	11쪽
R07	P1	배포 설정에 Redis 연결이 없어 제한 기능이 fail-open 가능	12쪽
R08	P1	운영 스키마를 앱 시작 시 자동 변경하며 변경 이력이 없음	13쪽
R09	P2	저장소의 외부 진입점에 HTTPS 강제 구성이 없음	14쪽
R10	P2	커밋 전 성공 로그와 부족한 감사 연결 정보	15쪽
R11	P2	영업일 파라미터가 대사 데이터의 기준 시점을 제한하지 않음	16쪽
R12	P2	대사 불일치를 로그로만 남겨 후속 조치와 알림이 누락	17쪽
R13	P2	배치 누락·중단 후 재시작을 보장하는 복구 경로가 없음	18쪽
R14	P2	배포할 SHA의 테스트 통과가 CD 의존성으로 묶이지 않음	19쪽
R15	P2	latest 적용 후 SHA 변경과 동시 배포가 릴리스 순서를 흔듦	20쪽
R16	P2	관리 포트만 검사해 실제 API 장애를 놓칠 수 있음	21쪽
R17	P2	동시 이체 테스트가 전건 실패도 통과할 수 있음	22쪽
R18	P2	멱등성 헤더의 빈 값·길이·충돌 종류를 검증하지 않음	23쪽
R19	P2	거래 이력 페이지에 고유한 정렬 기준이 없음	24쪽

P1 8건 중 R04는 실자금 전환 조건, R06은 프록시 입력 신뢰 조건, R07은 제공된 배포 설정 기준입니다. 조건부 위험을 실제 운영 사고로 표현하지 않았습니다.

R01 거래 커밋과 멱등성 완료 저장이 분리되어 중복 처리 가능

P1 | 거래 정합성 | 단위 E1 + 실제 MySQL 완료 UPDATE 장애 주입 D4

> 성공한 거래가 실패 응답으로 보인 뒤, 같은 Idempotency-Key의 재시도로 한 번 더 반영될 수 있다.

코드 위치: domain/idempotency/IdempotencyService.java:22-39 · domain/idempotency/IdempotencyService.java:52-56 · domain/transaction/TransactionService.java:45-54

원인

execute()에는 트랜잭션이 없다. 키 INSERT, action의 거래 트랜잭션, 완료 응답 저장은 별도 단위다. action.get()이 정상 반환하면 서비스 프록시의 커밋은 이미 끝났는데, complete() 실패까지 같은 catch에서 받아 키를 삭제한다. 프로세스가 그 사이 종료되면 IN_PROGRESS가 영구 잔류하는 반대 문제도 생긴다.

발생 조건과 영향

D4에서 실제 MySQL 트리거로 COMPLETED 갱신만 실패시켰다. 100 입금은 커밋되어 잔액 100·이력 1건이 남았으나 멱등성 키는 삭제됐다. 장애 제거 후 같은 키로 재시도하자 잔액 200·이력 2건이 됐다. 완료 기록 실패와 거래 커밋의 분리가 중복 반영으로 이어짐을 실측했다. 프로세스 강제 종료는 검증하지 않았다.

```
T1: save(IN_PROGRESS) -> COMMIT
T2: balance + history -> COMMIT
T3: save(COMPLETED) -> ERROR
catch: delete(key) -> retry executes T2 again
```

최소 수정 방향

키 선점, 잔액 변경, 거래 이력, 완료 결과를 동일 DB 트랜잭션에서 확정하는 파사드를 둔다. 재시도는 이 전체 트랜잭션의 바깥에서 수행한다. 유니크 충돌은 실패한 트랜잭션을 종료한 뒤 새 조회로 처리한다. 단순히 현재 execute()에 @Transactional만 추가하면 내부 재시도와 rollback-only 경계가 얽힐 수 있으므로 피한다.

수정 완료 기준

완료 저장 실패·직렬화 실패·커밋 직후 응답 유실·동일 키 50개 동시 요청을 MySQL에서 검증한다. 승인된 단일 요청의 거래 이력과 잔액 변경이 정확히 1회이며 재요청의 응답이 같은지 확인한다.

R02 멱등성 키에 사용자·거래 종류·요청 내용이 결합되지 않음

P1 | 인가 / 멱등성 | 단위 E2 + 실제 MySQL / 인증 필터 포함 MockMvc D2-D3

> 동일 키를 아는 다른 사용자가 기존 거래 응답을 받거나, 다른 금액의 요청이 실행되지 않았는데 성공처럼 보일 수 있다.

코드 위치: domain/idempotency/IdempotencyKey.java:29-40 · domain/idempotency/IdempotencyService.java:43-49 · domain/transaction/TransactionController.java:40-42

원인

유니크 제약은 key 하나에만 적용되고 requesterId, operation, requestHash가 없다. 기존 키 발견 시 action을 실행하지 않으므로 action 내부의 계좌 소유자 검증도 생략된다. 입금과 출금은 같은 DTO를 써서 경로가 달라도 기존 응답이 반환될 수 있다.

발생 조건과 영향

D2: A가 키 K로 100 입금 후 B가 A 계좌 출금을 요청했다. 새 키는 403이었으나 K 재사용은 200이며 A의 계좌·잔액 100·DEPOSIT 응답이 노출됐다. 실제 출금은 없었다. D3: 같은 사용자가 K로 금액을 100→200으로 바꿔도 200 응답과 기존 금액 100을 받았다. 키를 알아야 하는 조건은 있으나 키가 인가 경계를 대신할 수는 없다.

```
execute(key, responseType, action)
// no requesterId, operation, requestHash
return deserialize(existing.getResponseBody(), responseType);
```

최소 수정 방향

인증 주체와 작업 종류를 포함한 유니크 키를 설계하고, 정규화한 계좌·금액·통화의 requestHash를 저장한다. 재생 전 소유 범위와 해시를 비교하고 다른 요청이면 명시적인 409 오류를 반환한다. 응답에는 추적 가능한 거래 식별자를 포함한다.

수정 완료 기준

서로 다른 사용자/작업/금액/수취계좌의 키 재사용을 모두 검증한다. 타 사용자 응답 노출은 0건, 동일 사용자·동일 요청만 재생 가능해야 한다. D2-D3의 현재 위험 동작을 수정 후 회귀 테스트에서 차단해야 한다.

R03 금액 scale·상한 미검증으로 이체 시 총액 보존 위반

P1 | 금액 불변식 | 입력 검증 E3 + 실제 MySQL 8.0.46 / MockMvc 재현 D1

> 0.015 이체가 HTTP 200으로 승인되고 두 계좌 합계가 1.00에서 1.01로 증가했다. 응답은 0.015, 이력 저장 금액은 0.02로 달랐다.

코드 위치: domain/transaction/dto/DepositReqDto.java:8-11 · domain/transaction/dto/WithdrawReqDto.java:8-11 · domain/transaction/dto/TransferReqDto.java:16-18 · domain/account/Account.java:38-39 · domain/account/AccountRepository.java:21-38

원인

@DecimalMin은 양수 하한만 검사한다. DECIMAL(19,2)의 정밀도와 일치하는 @Digits 및 거래 한도가 없다. DB에서 양쪽 잔액과 거래 이력을 각각 반올림하면 이체 총액 보존과 원장 합계가 어긋날 수 있다. 응답 DTO의 amount는 요청 BigDecimal을 그대로 사용한다.

발생 조건과 영향

D1에서 A=1.00, B=0.00으로 시작해 실제 이체 API 경로로 0.015를 보냈다. DB 재조회 결과 A=0.99, B=0.02, 합계=1.01이었다. TRANSFER 이력 1건의 amount는 0.02였고 HTTP 응답은 0.015였다. 이 결과는 반올림 규칙 추정이 아닌 MySQL 실측이다. 과대 금액은 DTO 검증 통과까지 확인했다.

```
@DecimalMin("0.01") BigDecimal amount;
@Column(precision = 19, scale = 2) BigDecimal balance;
// input precision and persisted precision differ
```

최소 수정 방향

통화 단위를 먼저 확정한다. 현재 2자리 정책이면 @Digits(integer=17, fraction=2), 업무상 1회/일 한도, 잔액 상한을 적용한다. KRW 정수 정책이면 fraction=0으로 일관되게 바꾼다. 서비스 경계에서도 금액을 검증하고, 묵시적 반올림 대신 허용 불가 금액을 거절한다.

수정 완료 기준

0, 음수, 0.001, 0.015, 1.999, 최대 허용액, 잔액 overflow를 MySQL로 확인한다. 거래 전후 총액 보존, 이력 합계 일치, HTTP 응답 금액 일치를 함께 검사한다. [S1]

R04 일반 사용자 입금에 자금 유입 증빙·권한 경계가 없음

P1 | 실자금 전환 필수 통제 | 정적 확인 / 학습용 모의 자금이라는 현재 맥락 반영

> 가입한 사용자는 자신의 계좌에 원하는 양수 금액을 반복 입금하고 다른 계좌로 이체할 수 있다.

코드 위치: domain/transaction/TransactionController.java:33-43 · domain/transaction/TransactionService.java:47-54 · common/security/SecurityConfig.java:35-42

원인

로그인 및 계좌 소유 여부만으로 잔액 증가를 허용한다. 외부 입금 확인, 승인된 내부 운영자, 지급기관 거래번호, 결제 확정 상태가 없다. 이는 학습용 시뮬레이터에서는 가능한 요구사항이지만 실자금 계정계의 입금 승인 조건으로는 부족하다.

발생 조건과 영향

회원가입 → 계좌개설 → 새로운 멱등성 키로 큰 금액 입금 → 타인 계좌 이체가 모두 공개 API 흐름에 존재한다. 실제 자금의 유입 없이 고객 원장을 증가시키므로 실자금 서비스로 연결하면 무담보 잔액이 생성된다.

```
validateOwner(account, requesterId);
increaseBalance(account, amount);
// ownership is not proof of incoming funds
```

최소 수정 방향

모의 서비스라면 해당 API를 명확히 모의 충전 기능으로 명명하고 prod에서는 비활성화한다. 실입금은 신뢰된 외부 입금 이벤트 또는 승인된 업무 API만 수행하도록 분리하고, 외부 거래번호의 유니크 제약과 승인 증빙을 같은 거래 기록에 연결한다.

수정 완료 기준

일반 사용자 토큰으로 실제 입금 처리 API를 호출하면 거절되는지 확인한다. 승인되지 않은 이벤트, 중복 외부 거래번호, 금액/통화 불일치가 잔액을 바꾸지 않아야 한다.

R05 검증 예외 원문 로그에 비밀번호 등 거절된 값이 노출

P1 | 민감정보 보호 | 실제 예외 핸들러 로그 캡처 재현 E4

> 8자 미만 비밀번호로 회원가입을 요청하면 해당 평문이 WARN 로그에 남을 수 있다.

코드 위치: common/exception/GlobalExceptionHandler.java:21-34 · domain/member/dto/MemberCreateReqDto.java:7-10 · src/main/resources/logback-spring.xml:5-11

원인

MethodArgumentNotValidException.getMessage()에는 FieldError의 rejected value가 들어간다. 핸들러가 전체 메시지를 로그로 기록하므로 AccountNumberMasker와 무관하게 민감정보가 빠져나간다. JSON 포맷은 구조화만 제공하며 값의 마스킹을 보장하지 않는다.

발생 조건과 영향

E4에서 가짜 테스트 비밀번호 probe7을 거절 값으로 만든 예외를 실제 핸들러에 전달하고 로그를 수집했다. 결과에 rejected value [probe7]이 포함되었다. 형변환·JSON 파싱 예외 원문도 입력값을 포함할 가능성이 있어 같은 기준으로 점검해야 한다.

```
log.warn("Validation failed: {}", e.getMessage());
// FieldError text includes rejected value
```

최소 수정 방향

로그에는 requestId, 오류 코드, 필드 이름, 검증 규칙만 허용 목록 방식으로 남긴다. 입력값·DTO toString·예외 메시지를 직접 남기지 않는다. 예외 스택은 민감값이 포함되지 않도록 별도 정책을 적용한다. 성공 거래 로그의 금액도 접근 범위와 보존 정책을 정한다.

수정 완료 기준

비밀번호, 이메일, 계좌번호, 토큰을 포함한 유효성/파싱 실패를 발생시키고 로그 전체에서 원문이 검색되지 않는지 검사한다. HTTP 오류 응답도 함께 확인한다. [S2]

R06 X-Forwarded-For를 무조건 신뢰하여 요청 제한 우회 가능

P1 | 로그인 보호 / 프록시 신뢰 | 필터 동작 재현 E5 / 실제 외부 악용은 인그레스 설정에 의존

> 클라이언트가 헤더를 바꿀 때마다 다른 로그인 버킷을 사용할 수 있다.

코드 위치: common/ratelimit/RateLimitFilter.java:63-71 · common/ratelimit/RateLimitFilter.java:74-77 · k8s/ingress.yaml:1-15

원인

연결한 프록시가 신뢰된 대상인지 검사하지 않고 X-Forwarded-For의 첫 값을 Redis 키로 사용한다. IP 형식·길이 검증도 없다. 저장소의 Ingress 설정에는 원본 헤더 제거/덮어쓰기 및 신뢰 프록시 설정이 포함되어 있지 않다.

발생 조건과 영향

E5에서 같은 remoteAddr를 가진 두 요청에 서로 다른 헤더를 넣자 서로 다른 로그인 키를 소비했다. 직접 앱에 연결 가능하거나 인그레스가 임의 헤더를 보존하는 환경에서는 로그인 제한을 우회할 수 있다. 외부 프록시가 이를 안전하게 정규화한다면 위험도는 낮아지므로 실제 설정 확인이 필요하다.

```
String forwardedFor = request.getHeader("X-Forwarded-For");
return forwardedFor.split(",")[0].trim();
```

최소 수정 방향

신뢰 프록시 목록과 헤더 처리 정책을 한 곳에 정의한다. 인터넷 입력 헤더를 경계 프록시에서 제거/정규화하고 앱 직접 접근을 차단한다. 로그인에는 IP뿐 아니라 계정 단위 제한과 지연 정책을 함께 적용한다.

수정 완료 기준

실제 인그레스를 통과하는 요청으로 헤더 위조를 시도해 한도가 유지되는지 검증한다. 내부 직접 접근 차단, 여러 프록시 체인, NAT 사용자, 잘못된 IP 문자열도 확인한다.

R07 배포 설정에 Redis 연결이 없어 제한 기능이 fail-open 가능

P1 | 배포 / 보안 의존성 | 배포 산출물 교차 확인 / 실행 중 클러스터는 미확인

> 제공된 매니페스트 그대로 배포하면 Redis는 각 앱 파드의 localhost:6379를 바라보며, 실패한 요청 제한은 허용으로 처리된다.

코드 위치: src/main/resources/application.yaml:26-29 · .github/workflows/cd.yml:41-53 · k8s/deployment.yaml:29-56 · common/ratelimit/TokenBucketRateLimiter.java:68-83

원인

CD Secret에 DB와 JWT 값만 있고 Redis 주소는 없다. Deployment에도 Redis sidecar나 주소 주입이 없다. Redis 접속 실패는 tryConsume에서 모든 요청 허용으로 전환되며 로그인에도 같은 정책이 적용된다. 요청마다 실패한 접속을 기다리고 경고를 기록해 지연과 로그도 증가할 수 있다.

발생 조건과 영향

저장소에 정의된 환경만으로 앱 2개 파드를 띄운다. 같은 파드 안에 Redis가 없으면 요청마다 Redis 오류가 발생하고 rate limit이 작동하지 않는다. 별도 admission 주입이나 외부 설정이 있다면 달라질 수 있으나 저장소에서는 확인되지 않는다.

```
host: ${REDIS_HOST:localhost}
catch (Exception e) {
 return true; // shared by login and general APIs
}
```

최소 수정 방향

Redis 서비스 주소와 필요한 인증/TLS 설정을 배포 구성에 명시한다. 연결·명령 timeout을 제한하고 가용성 지표를 추가한다. 로그인 장애 정책은 일반 조회와 분리해 제한된 로컬 fallback 또는 명시적 일시 거절 등으로 결정한다. 무조건적인 무제한 허용은 제거한다.

수정 완료 기준

prod 설정으로 같은 IP의 로그인 한도를 넘겨 429가 나오는지 검증한다. Redis 중단 시 보안 정책, p99 지연, 로그량, 복구 후 제한 동작을 확인한다.

R08 운영 스키마를 앱 시작 시 자동 변경하며 변경 이력이 없음

P1 | DB 변경 통제 | 설정·의존성·CD 코드 정적 확인

> prod에서도 기본 ddl-auto:update가 적용되며, 스키마 변경과 Batch 메타테이블 준비가 재현 가능한 배포 단계로 관리되지 않는다.

코드 위치: src/main/resources/application.yaml:14-21 · src/main/resources/application.yaml:31-39 · .github/workflows/cd.yml:47-52 · build.gradle:18-49

원인

별도 tracked prod 설정이나 Flyway/Liquibase 마이그레이션이 없다. CD는 DB 사용자명 admin을 사용한다. 이름만으로 실제 권한을 단정할 수는 없지만 실행 계정과 DDL 계정의 분리가 코드에 나타나지 않는다. 새 DB에서는 수동 Batch 스키마 생성이 누락되면 배치가 실패한다.

발생 조건과 영향

두 파드가 롤링 업데이트되는 동안 새 엔티티 매핑이 자동 DDL을 유발하면 검토되지 않은 구조 변경과 테이블 잠금이 발생할 수 있다. 배포 실패 후 코드만 되돌려도 DB 스키마는 이전 상태로 돌아가지 않는다.

```
spring.jpa.hibernate.ddl-auto: update
// no tracked versioned migrations
// Batch schema is a manual prerequisite
```

최소 수정 방향

운영은 ddl-auto:validate로 제한하고 버전 관리되는 마이그레이션을 별도 단계에서 실행한다. 기존 DB baseline과 Batch 스키마를 포함한다. expand-contract 방식으로 신구 버전 호환을 유지하고, 런타임 계정에는 필요한 DML 권한만 부여한다.

수정 완료 기준

빈 DB 설치, 현재 운영 스키마에서 업그레이드, 이전 앱과 신규 앱 동시 실행, DDL 권한 없는 계정 기동을 검증한다. 실제 데이터 복원 및 롤백 절차를 준비한다.

R09 저장소의 외부 진입점에 HTTPS 강제 구성이 없음

P2 | 전송 구간 보호 | 설정 누락 확인 / 외부 TLS 종료 장비는 확인 필요

> 매니페스트만으로는 회원 비밀번호와 Bearer 토큰이 HTTPS로만 전달된다고 보장할 수 없다.

코드 위치: k8s/ingress.yaml:5-15 · common/security/SecurityConfig.java:31-43

원인

Ingress에 host, tls, HTTP→HTTPS 전환 정책이 없다. 코드 저장소 밖의 ALB/Traefik 전역 설정이 보호할 가능성은 있으므로 실제 평문 노출이 발생했다고 단정하지 않는다.

발생 조건과 영향

외부 TLS 종료 설정 없이 이 Ingress를 그대로 사용하면 HTTP 로그인 및 거래 경로가 열린다. 토큰 탈취는 인증과 계좌 소유 검사 자체를 우회하는 효과를 낸다.

```
spec:
 rules:
 - http: ...
# no host / tls / redirect policy in this manifest
```

최소 수정 방향

TLS 종료 위치를 아키텍처와 배포 구성에 명시하고 인증서, 호스트 검증, HTTPS 강제와 필요한 HSTS 정책을 적용한다. 우회 가능한 직접 앱/노드 포트도 차단한다. 외부 장비가 담당한다면 해당 설정을 검토 가능한 증거로 연결한다.

수정 완료 기준

HTTP 로그인 요청이 애플리케이션에 도달하지 않는지, HTTPS 인증서 및 호스트 검증이 정상인지, 인증서 만료를 감지하는지 확인한다.

R10 커밋 전 성공 로그와 부족한 감사 연결 정보

P2 | 감사 추적 | 정적 호출 순서·엔티티 컬럼 확인

> 로그의 "입금/출금/이체 완료"가 DB 확정 이전에 기록되며, 보안 감사와 거래를 안정적으로 연결하기 어렵다.

코드 위치: domain/transaction/TransactionService.java:51-54 · domain/transaction/TransactionService.java:118-122 · domain/transaction/TransactionHistory.java:30-59

원인

@Transactional 프록시는 메서드 반환 후 커밋하므로 메서드 내부 성공 로그 뒤에 커밋 실패가 가능하다. TransactionHistory에는 수행자, 요청/멱등성 식별자, 원거래 참조가 없다. 일반 application 로그와 거래 이력만으로 누가 어떤 승인 근거로 처리했는지 충분히 추적하기 어렵다.

발생 조건과 영향

이력 저장 후 성공 로그를 남긴 직후 커밋이 실패하면 실제 잔액은 반영되지 않았는데 운영자는 완료 로그를 본다. 재시도 로그와 최종 거래를 동일한 거래 ID로 연결할 수도 없다.

```
save(history);
log.info("이체 완료 ...");
return response;
// transaction proxy commits after this return
```

최소 수정 방향

확정 성공 이벤트는 afterCommit 또는 트랜잭션과 함께 저장한 감사 레코드를 기준으로 생성한다. 실패 시도와 확정 거래를 구분하고 transactionId, requesterId, operation, requestId, idempotency 참조를 남긴다. 수정·삭제 권한과 보존 정책은 별도 운영 통제로 제한한다.

수정 완료 기준

커밋 실패 때 확정 성공 이벤트가 0건인지 확인한다. 한 거래의 요청·실패 시도·최종 성공을 식별자로 추적하고, 원장 변경 권한의 최소 범위를 점검한다.

R11 영업일 파라미터가 대사 데이터의 기준 시점을 제한하지 않음

P2 | 일 마감 / 정산 의미 | 배치 파라미터 사용처 및 SUM 쿼리 정적 확인

> 전일 businessDate로 실행하지만 실제로는 실행 중 각 청크 시점의 현재 잔액과 전체 기간 이력을 비교한다.

코드 위치: domain/reconciliation/BalanceReconciliationScheduler.java:43-46 · domain/reconciliation/BalanceReconciliationProcessor.java:20-26 · domain/transaction/TransactionHistoryRepository.java:27-31 · domain/reconciliation/BalanceReconciliationJobConfig.java:62-65

원인

businessDate는 JobInstance 식별에만 사용되고 reader/processor/query에 전달되지 않는다. created_at cutoff와 일별 확정 잔고가 없다. 따라서 현재 잔액 정합성 점검으로는 의미가 있으나 전일 마감 결과를 재생산하는 배치는 아니다.

발생 조건과 영향

9월 28일 대사를 29일 01시에 실행하면 29일 00:30 거래도 합산된다. 같은 28일 업무를 며칠 후 재실행하면 또 다른 전체 기간 데이터가 검증된다. 청크별 스냅샷도 서로 다른 시점이므로 하루 전체의 단일 마감 기준은 아니다.

```
addLocalDate("businessDate", yesterday)
calculateNetAmount(accountId)
// query has no cutoff or businessDate condition
```

최소 수정 방향

현재 목적이 상시 잔액 대사라면 명칭과 파라미터 의미를 그에 맞게 바꾼다. 일 마감이 목적이면 타임존과 cutoff를 정의하고 동일 기준시점의 잔액 스냅샷과 확정 거래 집계를 비교한다. 쿼리에 날짜 조건만 붙여 현재 잔액과 비교하는 수정은 피한다.

수정 완료 기준

자정 전후 거래, 마감 중 신규 거래, 지연 도착 거래, 과거 영업일 재실행을 검증한다. 같은 마감 데이터는 동일 결과를 내야 한다. 청크 스냅샷 가정은 실제 MySQL 격리 수준에서도 확인한다.

R12 대사 불일치를 로그로만 남겨 후속 조치와 알림이 누락

P2 | 재무 이상 탐지 | writer·scheduler·PrometheusRule 교차 확인

> 불일치 계좌가 발견돼도 Job은 COMPLETED가 될 수 있고, 현재 경보에는 해당 불일치 조건이 없다.

코드 위치: domain/reconciliation/BalanceReconciliationJobConfig.java:69-74 · domain/reconciliation/BalanceReconciliationScheduler.java:47-51 · k8s/prometheusrule.yaml:11-44

원인

writer는 WARN 로그만 남긴다. 스케줄러는 BatchStatus만 검사하며 재무 판정 결과를 검사하지 않는다. Prometheus 경보는 HTTP 오류율·지연·파드 재시작뿐이다. 자동 보정하지 않는 결정은 적절하지만 조사 건의 지속적 관리가 빠져 있다.

발생 조건과 영향

불일치 1건을 발견해 로그를 쓰고 배치가 성공 종료한다. 로그를 보지 않으면 인지하지 못하며 파드/로그 수집 장애 시 조사 근거가 부족해진다. 기존 통합테스트도 writeCount=1과 COMPLETED를 정상 결과로 검증한다.

```
writer: log.warn("잔고 불일치 발견 ...")
job status: COMPLETED
alerts: 5xx / p99 / pod restart
```

최소 수정 방향

runId/영업일/계좌 내부 ID/차이/탐지시각/조치상태를 가진 대사 결과를 권한 제한 저장소에 남긴다. 기술적 완료와 재무 정상 여부를 분리하고 mismatchCount 및 마지막 성공/완료 시각으로 알림을 연결한다. 자동 잔액 수정은 하지 않는다.

수정 완료 기준

불일치 주입 시 결과 레코드와 경보가 생성되고, 재실행에서 중복 조사 건이 제어되는지 확인한다. 0건 정상과 미실행/실패를 구분해야 한다.

R13 배치 누락·중단 후 재시작을 보장하는 복구 경로가 없음

P2 | 운영 복구 | 스케줄러 코드 및 명시된 한계 확인

> 01시에 모든 파드가 내려가 있거나 실행 중 프로세스가 종료되면 해당 영업일 검증이 누락될 수 있다.

코드 위치: domain/reconciliation/BalanceReconciliationScheduler.java:34-46 · domain/reconciliation/BalanceReconciliationScheduler.java:52-56

원인

@Scheduled는 그 순간 호출만 수행한다. 놓친 영업일을 따라잡는 실행과 수동 재실행 절차가 없다. 실행 중 종료되어 STARTED가 남는 위험은 주석에도 인정되어 있지만 복구 수단은 구현되어 있지 않다. 예외는 로그 후 종료된다.

발생 조건과 영향

배치 실행 중 파드가 종료되어 STARTED가 남는다. 같은 영업일은 already-running으로 차단되고, 다음 날 스케줄은 다른 날짜로 진행한다. 또는 01시 전체 장애 뒤 01:10 복구해도 그날 스케줄은 자동 보충되지 않는다.

```
@Scheduled(cron = "0 0 1 * * *", zone = "Asia/Seoul")
// no catch-up for missed business dates
```

최소 수정 방향

영업일별 완료 여부와 최근 실행 지연을 감시한다. 승인된 운영 명령으로 특정 날짜를 재실행하고, 오래된 STARTED는 실제 실행 주체가 없는지 확인한 뒤 안전하게 정리한다. 단일 스케줄러 또는 배치 오케스트레이션 도입은 이 복구 계약을 기준으로 결정한다.

수정 완료 기준

01시 미기동, 청크 중간 종료, 재시작, 동일 날짜 중복 실행을 검증한다. 누락된 영업일이 보충되고 중복 처리 없이 결과가 남아야 한다.

R14 배포할 SHA의 테스트 통과가 CD 의존성으로 묶이지 않음

P2 | 릴리스 품질 게이트 | CI/CD 트리거와 Docker build task 정적 확인

> master에 push된 커밋은 CD에서 테스트 실행 여부와 무관하게 이미지 빌드·배포된다.

코드 위치: .github/workflows/ci.yml:3-5 · .github/workflows/cd.yml:3-14 · Dockerfile:9-12

원인

CI는 PR만 대상으로 하고 CD는 master push에서 별도로 실행된다. Dockerfile은 bootJar만 실행하며 test를 포함하지 않는다. GitHub branch protection이 별도로 모든 push를 제한할 수는 있지만 해당 설정은 이번 검토 범위에 없다.

발생 조건과 영향

직접 push, 보호 규칙 예외, merge 결과와 PR 테스트 커밋의 차이 등으로 테스트를 통과하지 않은 SHA가 배포될 수 있다. 배포 성공은 테스트 성공을 의미하지 않는다.

```
CI: pull_request -> test
CD: push master -> docker build -> deploy
Docker: ./gradlew bootJar --no-daemon
```

최소 수정 방향

동일 SHA의 테스트→이미지 생성→스테이징 확인→배포를 명시적 job 의존성으로 묶는다. 또는 성공한 검증 워크플로의 검증된 SHA만 배포하도록 한다. 브랜치 보호와 운영 환경 승인 설정은 별도 증거로 관리한다.

수정 완료 기준

의도적으로 실패하는 테스트를 가진 커밋으로 배포 경로를 검증한다. 테스트 실패 시 이미지 승격/운영 변경이 0건이며 배포 기록에 검증 SHA가 남아야 한다.

R15 latest 적용 후 SHA 변경과 동시 배포가 릴리스 순서를 흔듦

P2 | 배포 재현성 | 매니페스트 이미지와 CD 순서 정적 확인

> 한 릴리스에서 latest 파드와 특정 SHA 파드가 연속으로 생성되거나, 오래된 실행이 새 버전을 덮어쓸 수 있다.

코드 위치: k8s/deployment.yaml:23-24 · .github/workflows/cd.yml:27-31 · .github/workflows/cd.yml:55-64

원인

kubectl apply로 latest를 지정한 Deployment를 먼저 적용하고 이어서 set image로 SHA를 바꾼다. CD 워크플로에는 concurrency 직렬화가 없다. 두 push의 build/deploy 완료 순서는 commit 순서와 다를 수 있다.

발생 조건과 영향

현재 A 버전에서 B 배포 시 apply가 latest 롤아웃을 유발하고 다음 명령이 B로 다시 바꾼다. B와 C가 겹치면 늦게 끝난 B 실행이 C 이후 B 이미지를 다시 설정할 수 있다. 실제 발생 여부는 실행 순서에 의존한다.

```
kubectl apply -f deployment.yaml # image: latest
kubectl set image ...:${github.sha}
// workflow has no environment concurrency group
```

최소 수정 방향

대상 SHA 또는 image digest를 매니페스트에 렌더링해 한 번만 적용한다. 환경 단위 배포를 직렬화하고 운영 반영 전에 최신 허용 릴리스인지 검사한다. 중간 실행을 취소할 때도 부분 배포 상태가 남지 않도록 한다.

수정 완료 기준

두 릴리스를 연속 실행하고 오래된 build를 지연시켜 최종 버전이 역전되지 않는지 확인한다. rollout 중 모든 pod의 image digest와 배포 기록을 대조한다.

R16 관리 포트만 검사해 실제 API 장애를 놓칠 수 있음

P2 | 가용성 / 헬스체크 | 프로브 설정 + Spring Boot 공식 동작 설명

> 관리 서버 9090은 응답하지만 서비스 서버 8080이 요청을 처리하지 못하는 상황에서도 파드가 Ready일 수 있다.

코드 위치: k8s/deployment.yaml:42-53 · src/main/resources/application.yaml:68-79

원인

readiness/liveness 모두 별도 management 포트를 검사한다. 기본 readiness 그룹에 업무 경로의 연결 풀·처리 가능 상태가 자동으로 모두 포함되는 것도 아니다. 외부 의존성을 liveness에 무조건 추가하면 대규모 재시작을 유발할 수 있어 구분이 필요하다.

발생 조건과 영향

8080 처리 자원이 고갈되어 고객 요청은 대기하지만 9090 probe가 계속 성공한다. Service는 해당 파드로 요청을 전달한다. 현재는 실제 요청 경로 장애 주입 검증이 없다.

```
management.server.port: 9090
readinessProbe.httpGet.port: 9090
livenessProbe.httpGet.port: 9090
```

최소 수정 방향

메인 서버에 추가 probe 경로를 노출하고 readiness로 실제 요청 수신 경로를 검증한다. liveness는 프로세스 자체 복구 필요 여부, readiness는 트래픽 수신 가능 여부로 나눈다. DB/Redis 장애의 전파 정책은 명시적으로 정하고 startupProbe도 기동시간에 맞게 검토한다.

수정 완료 기준

8080만 장애, DB 지연, Redis 지연, 긴 초기화에서 라우팅과 재시작 동작을 확인한다. 공유 DB 장애가 전체 파드 재시작 폭풍으로 이어지지 않아야 한다. [S3]

R17 동시 이체 테스트가 전건 실패도 통과할 수 있음

P2 | 테스트 신뢰도 | 테스트 assertion 및 예외 처리 정적 확인

> 양방향 이체가 모두 CONCURRENT_UPDATE_CONFLICT로 실패해도 합계 잔액만 유지되면 테스트가 통과할 수 있다.

코드 위치: src/test/java/com/trading/account/domain/transaction/TransactionConcurrencyTest.java:125-145 · src/test/java/com/trading/account/domain/idempotency/IdempotencyServiceTest.java:39-97 · src/test/java/com/trading/account/domain/member/MemberRepositoryTest.java:11-12

원인

runTransfer는 충돌 소진 오류를 수집하지 않고 성공 건수도 세지 않는다. 최종 검증은 완료 시간·예상 밖 오류·두 계좌 합계뿐이다. 원본 저장소의 멱등성 테스트는 Mockito 단위 수준이다. 감사용 D1-D5는 격리 복사본에만 있어 원본 CI의 회귀 안전장치가 아니다. MemberRepositoryTest는 H2 경로다.

발생 조건과 영향

이번 기존 동시성 테스트는 통과했다. 그러나 모든 이체가 충돌 오류를 반환하는 경우에도 unexpectedFailures는 비어 있고 A+B는 초기값 그대로라 현재 assertion을 만족한다. 이번 실행에서 전건 실패했다고 주장하는 것이 아니라, 그 상태를 구별할 검증 조건이 없다는 지적이다.

```
if (error != CONCURRENT_UPDATE_CONFLICT) failures.add(e);
assertThat(balanceA.add(balanceB)).isEqualTo(initialTotal);
// no successful-transfer count assertion
```

최소 수정 방향

성공/업무 거절/충돌 소진을 각각 집계하고 허용 성공률·지연 기준을 정한다. 성공 건수와 거래 이력 수, 각 계좌 순변동을 함께 검증한다. DB 멱등성·롤백·타 사용자 접근 테스트를 추가하고 MySQL 의미가 중요한 repository 검증을 실제 MySQL로 통일한다.

수정 완료 기준

일부러 전건 충돌을 반환시키는 변형에서 테스트가 실패해야 한다. 소수 금액·자금 부족 경합·이력 저장 실패·최종 커밋 실패·JWT 만료/위변조도 필수 회귀 목록에 포함한다.

R18 멱등성 헤더의 빈 값·길이·충돌 종류를 검증하지 않음

P2 | API 입력 / 장애 응답 | Controller·엔티티 제약·예외 분기 정적 확인

> 빈 키는 여러 요청을 하나로 묶고, 100자 초과 키의 DB 오류는 중복 키처럼 처리될 수 있다.

코드 위치: domain/transaction/TransactionController.java:34-42 · domain/idempotency/IdempotencyKey.java:32-33 · domain/idempotency/IdempotencyService.java:23-26

원인

@RequestHeader는 헤더 존재를 요구하지만 값의 유효성까지 정의하지 않는다. 엔티티 저장 길이는 100이며, 모든 DataIntegrityViolationException을 중복으로 분류한다. 길이 초과가 이 예외로 번역되면 해당 키가 없어 내부 오류를 반환한다.

발생 조건과 영향

공백 키 요청이 허용되면 서로 다른 정상 거래가 같은 키로 충돌한다. 긴 키는 400이 아닌 500 경로가 될 수 있다. 보존/만료/정리 정책도 없어 키 테이블이 지속 증가한다.

```
@RequestHeader("Idempotency-Key") String key
@Column(length = 100, unique = true)
catch (DataIntegrityViolationException e) { replayOrReject(...); }
```

최소 수정 방향

공백 금지, 최대 길이, 허용 문자 또는 UUID 정책을 API 경계에서 명시한다. DB 제약 이름/원인을 확인해 유니크 충돌만 replay로 처리한다. 보존 기간은 업무 재시도 기간과 맞추고, 삭제 후 키 재사용 및 거래 보관 정책을 함께 정의한다.

수정 완료 기준

헤더 없음, 빈 값, 공백, 100/101자, Unicode, 대소문자 구분, 서로 다른 무결성 오류를 검증한다. 유효하지 않은 입력은 업무 실행 전에 일관된 400으로 거절해야 한다.

R19 거래 이력 페이지에 고유한 정렬 기준이 없음

P2 | 조회 정확성 | Repository 쿼리 및 페이지 테스트 정적 확인

> createdAt이 같은 거래가 여러 개 있거나 조회 중 새 거래가 생기면 페이지에서 중복/누락을 볼 수 있다.

코드 위치: domain/transaction/TransactionHistoryRepository.java:16-22 · domain/transaction/TransactionController.java:71-78 · src/test/java/com/trading/account/domain/transaction/TransactionHistoryRepositoryTest.java:32-45

원인

ORDER BY createdAt DESC만 있어 동률을 결정하지 못한다. offset 기반 페이지는 신규 삽입에도 경계가 밀린다. 기존 테스트는 첫 페이지 총 개수만 확인한다. 이는 DB 거래 유실이 아니라 조회 결과의 안정성 문제다.

발생 조건과 영향

같은 타임스탬프의 거래가 페이지 경계에 걸리면 동일 쿼리의 후속 페이지에서 순서가 달라질 수 있다. 1페이지 조회 후 새 거래가 맨 앞에 들어오면 2페이지에서 이전 행을 다시 볼 수 있다.

```
ORDER BY t.createdAt DESC
// add a unique t.id tie-breaker
// use a cursor when stable traversal is required
```

최소 수정 방향

최소한 createdAt DESC, id DESC로 정렬을 고정하고 허용 정렬 필드와 page size를 제한한다. 일관된 연속 조회가 필요하면 두 값을 포함한 cursor와 조회 기준시점을 사용한다. 복합 인덱스는 실제 EXPLAIN과 데이터 분포를 보고 결정한다.

수정 완료 기준

동일 createdAt 다수, 페이지 사이 신규 삽입, 작은 page size로 전체 순회 시 중복/누락을 검사한다. 큰 데이터에서 실행 계획과 p95/p99를 측정한다.

개선의 중심은 하나의 원자적 처리 단위

현재 규모에서는 분산 락·Kafka·마이크로서비스를 먼저 늘릴 이유가 없습니다. 단일 MySQL에서 요청 계약과 트랜잭션 경계를 정확히 만드는 것이 우선입니다.

인증 및 요청 정규화
주체 · 작업 종류 · 계좌 · 통화 · 허용 자릿수 · requestHash를 확정

↓

재시도 경계: 전체 트랜잭션을 새로 시작
일시적 DB 충돌만 제한적으로 재시도 · 업무 실패는 즉시 반환

↓

동일 DB 트랜잭션
키 선점 + 조건부 잔액 변경 + 거래 이력 + 완료 결과 + 필요한 감사 기록

↓

커밋 후 응답 / 재생
동일 범위와 동일 해시만 저장 응답 재생 · 최종 transactionId 반환

함께 결정할 API 계약

계약	권장 완료 상태
같은 키 / 같은 요청	동일 거래 ID와 동일 확정 응답
같은 키 / 다른 요청	명시적 409, 거래 실행 0회
다른 사용자 / 같은 키	사용자 범위 분리, 타인 응답 노출 0건
완료 결과 저장 실패	거래까지 함께 롤백하거나 확정 상태를 안전하게 조회 가능
금액 정밀도 초과	API와 서비스에서 거절, DB 묵시적 반올림 금지

유니크 충돌 후 조회는 실패 트랜잭션 밖에서 처리합니다. Spring Retry와 Transactional 프록시 순서, 기존 트랜잭션 참여 여부까지 MySQL 통합테스트로 확인해야 합니다. [S4, S5]

전체 테스트와 단위 재현 결과

검증	관측 결과	한계
E1 / 완료 저장 실패	키 삭제 후 동일 키 action 2회 실행	mock 주입. 실제 DB 재현은 다음 쪽 D4.
E2 / 저장 응답 재생	action 내부 검증 없이 기존 응답 반환	사용자 교차 API 재현은 D2-D3.
E3 / 금액 검증	0.015 및 과대 금액 DTO 검증 통과	실제 소수 금액 이체 결과는 D1.
E4 / 검증 실패 로그	가짜 비밀번호가 로그에 포함됨	테스트용 입력만 사용.
E5 / 헤더 기반 버킷	같은 연결 IP에서 헤더별 다른 키 사용	외부 인그레스 헤더 처리 미확인.

기존 테스트: 22개 클래스 / 73개 전부 통과

기존 단위·standalone MVC 54개에 더해 전체 실행에서 19개를 확인했습니다. 앱 기동 1 · 회원 repository 2 · 계좌 repository 3 · 거래 이력 repository 2 · 전체 API 시나리오 5 · 대사 Job 1 · 거래 동시성 2 · 비관적 락 실험 1 · Redis 토큰 버킷 2입니다.

실행 단위와 결과

실행	결과
전체 test --rerun-tasks	기존 73 + 감사 E1-E5 5 = 78개, 실패/오류/skip 0. BUILD SUCCESSFUL, 2분 45초.
추가 DB 감사 클래스	D1-D5 5개 검증. 상세 수치와 장애 주입 방법은 다음 쪽.
테스트 종료 로그	공유 MySQL의 여러 JPA context 종료 시 FK drop 오류 로그가 있었음. 테스트 실패는 아니지만 종료 정리 소음은 개선 대상.

> 기존 테스트 73개가 통과해도 R01-R03의 위험은 남습니다. 감사 테스트는 현재 위험 동작을 관측하는 assertion이므로, 그 통과를 수정 완료나 실자금 운영 승인으로 해석하면 안 됩니다.

실제 MySQL에서 확인한 거래 경계

Docker 28.3.2 · MySQL 8.0.46(mysql:8.0) · redis:7-alpine. 원본 소스 수정과 운영 환경 접속 없이 테스트 DB만 초기화했습니다. D1-D3는 인증 필터를 포함한 MockMvc, D4-D5는 실제 서비스 프록시와 DB를 사용했습니다.

검증	실측 결과
D1 / 소수 금액 이체	A=1.00, B=0.00에서 0.015 이체 → HTTP 200. A=0.99, B=0.02, 총액 1.01. 이력 0.02 / 응답 0.015.
D2 / 사용자·작업 교차	A의 입금 키를 B가 A 계좌 출금 경로에 재사용. 새 키는 403, 기존 키는 200이며 A의 잔액 100과 DEPOSIT 응답 노출. 추가 출금은 없음.
D3 / 요청 금액 변경	같은 키로 입금 100 → 입금 200. 두 번째도 HTTP 200이나 응답 금액 100, 잔액 100, 이력 1건.
D4 / 완료 기록 실패	완료 UPDATE만 DB 오류로 거절. 입금은 잔액 100·이력 1건으로 커밋됐으나 키는 삭제됨. 같은 키 재시도 후 잔액 200·이력 2건.
D5 / 정상 상태 대조	동일 키로 20개 스레드 동시 호출. 완료/재생 1건, 처리 중 거절 19건. 최종 잔액 100, 거래 이력 1건: 이번 정상 DB 실험에서는 1회 반영.

D4 장애 주입과 검증 범위

Testcontainers MySQL에 임시 BEFORE UPDATE 트리거를 만들고 status=COMPLETED 때 SQLSTATE 45000을 발생시켰습니다. 테스트 메서드를 감싸는 트랜잭션 없이 커밋 결과를 재조회했습니다. 트리거는 finally에서 제거했습니다. 프로세스 종료·통신 단절 전체를 재현한 실험은 아닙니다.

재실행과 증거 보관

```
gradlew.bat --offline --no-daemon test --rerun-tasks
# Isolated review copy only:
gradlew.bat test --tests "*ReviewDatabaseEvidenceTest"
```

감사 클래스 ReviewEvidenceTest / ReviewDatabaseEvidenceTest는 작업용 복사본에만 있습니다. JUnit XML과 실행 로그를 작업 폴더에 보존했습니다. D4 첫 실행은 트리거 생성 권한 부족으로 실패하여, 테스트 컨테이너 관리자 연결로 트리거 생성/삭제만 수행하도록 보완 후 재실행했습니다. 앱 DB 계정·권한과 제품 코드는 변경하지 않았습니다.

수정 계획과 리뷰 승인 기준

단계	작업 묶음	승인에 필요한 증거
1 / 데이터 안전	R01-R03 멱등성·금액 불변식. 통화 단위와 요청 계약 확정.	MySQL에서 동일 키 동시 요청, 완료 저장/응답 유실 장애, 금액 경계 검증. 이력 1회·총액 보존.
2 / 보안·배포 기본	R04-R09 입금 권한, 비밀값 로그, 프록시·Redis, 마이그레이션, TLS.	일반 사용자 실입금 금지, 민감값 로그 0건, 배포 환경 429 확인, 버전 스키마 재현, HTTPS 우회 차단.
3 / 운영 신뢰	R10-R19 감사·대사·복구·배포 게이트·프로브·회귀 테스트·조회 계약.	중단된 배치 복구, 불일치 조사·알림, 실패 테스트 배포 차단, 릴리스 역전 방지, 실제 API 경로 probe.

실자금 서비스로 갈 때 별도 확인할 업무 계약

단일/다중 통화, 통화 최소 단위, 출금 가능 잔액과 보류 잔액, 계좌 활성/정지/해지, 입금 확인 주체, 거래 한도, 취소/역분개, 마감 기준시점, 거래 보존과 접근 권한을 문서화해야 합니다. 현재 요구사항에 없던 항목이므로 일괄 구현 버그로 계산하지 않았습니다.

운영 환경에서 받아야 할 증거

DB 사용자 권한 및 TLS 설정, backup/PITR와 복원 실험, 목표 RTO/RPO, 인그레스의 실제 헤더 처리, 네트워크 접근 정책, 알림 수신 확인, 브랜치 보호, image digest, 배포/롤백 실행 결과가 필요합니다.

수정 방식

데이터 정합성·보안·배포·배치의 수정은 작은 PR로 분리하되 R01과 재시도 경계는 함께 리뷰합니다. 각 PR에는 실패 재현, 원인, 수정 후 불변식 검증을 붙입니다. 수정하지 않은 조건부 위험은 책임자와 확인 증거를 남깁니다.

컴포넌트별 검토 결과

컴포넌트	유지할 점	남은 핵심 과제
account	소유자 검사, Luhn, 조건부 잔액 UPDATE	금액 서비스 경계, 번호 정규화 계약, 상한
transaction	한 트랜잭션에 잔액과 이력 저장, 갱신 순서	금액 정밀도, 거래 ID, 감사, 페이지 순서
idempotency	유니크 키, 처리 중/완료 구분	원자성, 주체/내용 범위, 입력·보존 정책
auth / member	해시 저장, 실패 응답 통일	로그 민감값, 입력 길이, 토큰 수명·폐기 정책
security	무상태 JWT와 서비스 소유권 확인	외부 TLS/프록시 증거, 토큰 검증 회귀
ratelimit	Redis Lua 원자 소비, 로그인 버킷 분리	Redis 배포, 신뢰 IP, 로그인 장애 정책
reconciliation	잔액 자동 덮어쓰기 없음, 청크 단위	일 마감 의미, 결과 보존, 누락·중단 복구
exception / logging	오류 코드와 상태 매핑, MDC 정리	원문 예외의 비밀값, 커밋 확정 로그
CI / CD / k8s	SHA 태그, 롤링 설정, 관리 포트 분리	테스트 gate, 배포 직렬화, 주 경로 probe
tests / docs	실DB 동시성 및 배치 테스트 존재	성공률·이력 assertion, 장애 경계, 현행화

무리한 구조 변경을 권하지 않는 이유

인터페이스 일괄 추가, 패키지 재배치, Kafka 도입, 전면 CQRS/샤딩 전환은 이번 핵심 문제의 직접 해법이 아닙니다. 현 단일 DB 구조에서도 대부분의 우선 문제를 해결할 수 있습니다. 성능 구조 변경은 실제 실행 계획과 부하 측정 이후 결정합니다.

차단 이슈와 구분한 유지보수 사항

1. 문서와 실제 동시성 방식 동기화

README, Account의 @Version 설명, 비관적 락 실험 주석은 과거 낙관적 갱신 흐름을 설명합니다. 현재의 조건부 원자적 UPDATE와 재시도 의미를 별도 "현재 구조" 절에 정리하고 과거 측정은 당시 커밋·환경·조건과 함께 남기는 편이 좋습니다.

2. 도메인 입력 방어

Account.deposit/withdraw 및 서비스 호출은 MVC 검증 밖에서도 실행할 수 있습니다. 현재 내부 호출자는 정상 금액을 전달하지만 양수·통화·정밀도 정책을 공유하는 도메인 검증을 두어 배치나 새 진입점에서 안전장치가 빠지지 않게 합니다. 회원 name/email/password에도 저장소와 인코더 한계에 맞는 입력 상한을 정합니다.

3. 토큰 처리와 수명 정책

JwtAuthenticationFilter는 validateToken 뒤 getMemberId에서 같은 토큰을 다시 파싱합니다. 한 번 검증·파싱한 결과를 사용하는 편이 간단하고 만료 경계 예외도 일관되게 다룰 수 있습니다.
issuer/audience, 키 회전, 탈퇴·정지·로그아웃 시 토큰 처리 여부는 제품 요구사항과 함께 정합니다.

4. 운영 설정과 용량

SQL DEBUG 로깅, OSIV 기본 동작, Redis timeout, connection pool, 배치 전체 이력 SUM, CPU request, 네트워크 정책, 컨테이너 실행 사용자, 의존성 점검은 운영 baseline에서 확인해야 합니다. 특정 규모에서 반드시 느리다고 단정하지 않고 EXPLAIN·부하·장애 측정으로 판단합니다.

5. 계좌번호와 설정 계약

Validator는 하이픈을 모두 제거해 검증하지만 조회는 원문 문자열을 사용합니다. 하이픈 없는 번호를 지원할지, 고정 형식만 허용할지 하나로 통일합니다. 토큰 버킷 속성에는 양수 검증을 추가하고, TTL은 capacity/refillTokens 비율이 달라져도 의도한 복구 시간을 유지하는지 확인합니다.

6. 문서 템플릿과 표준 명령

PR 템플릿에는 build.gradle에 없는 spotlessCheck 명령이 있고 이슈 템플릿 연락처는 placeholder입니다. 새 개발자가 바로 실행할 수 있는 local/CI/test 명령과 환경변수 이름을 README에 넣습니다. 모의 거래를 실서비스 능력으로 과장하는 문구는 정리합니다.

프로덕션 Java 검토 목록

공통 접두사: src/main/java/com/trading/account/ · 아래 파일은 모두 전체 내용을 읽고 호출 관계를 검토했습니다. 별도 지적이 없는 DTO·enum·기반 클래스도 포함됩니다.

파일 경로	줄 수
AccountApplication.java	26
common/exception/CustomException.java	14
common/exception/ErrorCode.java	31
common/exception/GlobalExceptionHandler.java	78
common/logging/AccountNumberMasker.java	16
common/logging/CorrelationIdFilter.java	44
common/ratelimit/RateLimitConfig.java	24
common/ratelimit/RateLimitFilter.java	79
common/ratelimit/RateLimitProperties.java	10
common/ratelimit/TokenBucketRateLimiter.java	86
common/response/ApiResponse.java	45
common/security/JwtAuthenticationEntryPoint.java	30
common/security/JwtAuthenticationFilter.java	37
common/security/JwtTokenProvider.java	62
common/security/SecurityConfig.java	46
domain/account/Account.java	71
domain/account/AccountController.java	39
domain/account/AccountNumberValidator.java	48
domain/account/AccountRepository.java	50
domain/account/AccountService.java	83
domain/account/dto/AccountBalanceResDto.java	9
domain/account/dto/AccountCreateResDto.java	13
domain/auth/AuthController.java	26
domain/auth/AuthService.java	35
domain/auth/dto/LoginReqDto.java	10
domain/auth/dto/LoginResDto.java	8

프로덕션 Java 검토 목록

공통 접두사: src/main/java/com/trading/account/ · 아래 파일은 모두 전체 내용을 읽고 호출 관계를 검토했습니다. 별도 지적이 없는 DTO·enum·기반 클래스도 포함됩니다.

파일 경로	줄 수
domain/idempotency/IdempotencyKey.java	55
domain/idempotency/IdempotencyKeyRepository.java	9
domain/idempotency/IdempotencyService.java	74
domain/idempotency/IdempotencyStatus.java	6
domain/member/Member.java	40
domain/member/MemberController.java	29
domain/member/MemberRepository.java	14
domain/member/MemberService.java	36
domain/member/dto/MemberCreateReqDto.java	12
domain/member/dto/MemberCreateResDto.java	11
domain/reconciliation/BalanceMismatch.java	10
domain/reconciliation/BalanceReconciliationJobConfig.java	76
domain/reconciliation/BalanceReconciliationProcessor.java	28
domain/reconciliation/BalanceReconciliationScheduler.java	59
domain/transaction/TransactionController.java	80
domain/transaction/TransactionHistory.java	61
domain/transaction/TransactionHistoryRepository.java	32
domain/transaction/TransactionService.java	169
domain/transaction/TransactionType.java	7
domain/transaction/dto/DepositReqDto.java	13
domain/transaction/dto/TransactionHistoryResDto.java	27
domain/transaction/dto/TransactionResDto.java	15
domain/transaction/dto/TransferReqDto.java	20
domain/transaction/dto/TransferResDto.java	12
domain/transaction/dto/WithdrawReqDto.java	13

테스트 Java 검토 목록

공통 접두사: src/test/java/com/trading/account/ · 모든 파일을 정적으로 검토했으며 실행 여부를 구분했습니다.
AbstractIntegrationTest와 실험 Service는 테스트 지원 코드입니다.

파일 경로	검증 범위
AbstractIntegrationTest.java	지원 코드
AccountApplicationTests.java	실행+검토
ScenarioIntegrationTest.java	실행+검토
common/exception/GlobalExceptionHandlerTest.java	실행+검토
common/logging/CorrelationIdFilterTest.java	실행+검토
common/ratelimit/RateLimitFilterTest.java	실행+검토
common/ratelimit/TokenBucketRateLimiterTest.java	실행+검토
common/response/ApiResponseTest.java	실행+검토
domain/account/AccountControllerTest.java	실행+검토
domain/account/AccountNumberValidatorTest.java	실행+검토
domain/account/AccountRepositoryTest.java	실행+검토
domain/account/AccountServiceTest.java	실행+검토
domain/idempotency/IdempotencyServiceTest.java	실행+검토
domain/member/MemberControllerTest.java	실행+검토
domain/member/MemberRepositoryTest.java	실행+검토
domain/member/MemberServiceTest.java	실행+검토
domain/reconciliation/BalanceReconciliationJobIntegrationTest.java	실행+검토
domain/reconciliation/BalanceReconciliationProcessorTest.java	실행+검토
domain/transaction/PessimisticLockExperimentTest.java	실행+검토
domain/transaction/PessimisticLockWithdrawService.java	지원 코드
domain/transaction/TransactionConcurrencyTest.java	실행+검토
domain/transaction/TransactionControllerTest.java	실행+검토
domain/transaction/TransactionHistoryRepositoryTest.java	실행+검토
domain/transaction/TransactionServiceTest.java	실행+검토

설정·배포·문서 검토 목록

Java 외 Git 추적 파일 30개. wrapper JAR는 바이너리 존재·해시만 확인. ignored application-local.yaml은 추가로 민감값을 제외하고 구조를 확인했습니다.

파일 경로	분류
.gitattributes	설정/배포
.github/ISSUE_TEMPLATE/bug_report.md	문서/정책
.github/ISSUE_TEMPLATE/config.yml	설정/배포
.github/ISSUE_TEMPLATE/feature_request.md	문서/정책
.github/pull_request_template.md	문서/정책
.github/workflows/cd.yml	설정/배포
.github/workflows/ci.yml	설정/배포
.gitignore	설정/배포
CLAUDE.md	문서/정책
Dockerfile	설정/배포
README.md	문서/정책
build.gradle	설정/배포
docs/CHAOS_AND_LOAD_TEST_SPEC.md	문서/정책
docs/LARGE_SCALE_TRAFFIC_SPEC.md	문서/정책
docs/PRODUCTION_HARDENING_SPEC.md	문서/정책
docs/PROJECT_SPEC.md	문서/정책
gradle/wrapper/gradle-wrapper.jar	바이너리
gradle/wrapper/gradle-wrapper.properties	설정/배포
gradlew	설정/배포
gradlew.bat	설정/배포
k8s/deployment.yaml	설정/배포
k8s/grafana-dashboard.yaml	설정/배포
k8s/ingress.yaml	설정/배포
k8s/prometheusrule.yaml	설정/배포
k8s/service.yaml	설정/배포
k8s/servicemonitor.yaml	설정/배포
settings.gradle	설정/배포
src/main/resources/application.yaml	설정/배포
src/main/resources/logback-spring.xml	설정/배포
src/test/resources/application.yaml	설정/배포

기술 근거와 참고 자료

외부 자료는 프레임워크 동작과 개선 방향의 근거에만 사용했습니다. 프로젝트 소스·비밀값을 검색 서비스에 전송하지 않았습니다. 확인일: 2026-09-29.

[S1] MySQL 8.0 - Rounding Behavior

DECIMAL 저장 시 반올림 규칙의 근거. R03은 D1에서 MySQL 8.0.46 실제 API 경로로 총액 증가와 응답/저장 금액 불일치를 확인했다.

https://dev.mysql.com/doc/refman/8.0/en/precision-math-rounding.html

[S2] OWASP - Logging Cheat Sheet

비밀번호·토큰·민감정보의 로그 제외와 안전한 로깅 원칙. R05의 노출 자체는 로컬 코드로 확인했다.

https://cheatsheetseries.owasp.org/cheatsheets/Logging_Cheat_Sheet.html

[S3] Spring Boot 3.4 - Actuator Endpoints

관리 포트와 실제 서비스 포트의 상태가 다를 수 있으며 메인 서버 probe 경로를 검토해야 한다는 근거.

https://docs.spring.io/spring-boot/3.4/reference/actuator/endpoints.html

[S4] Spring Data JPA - Transactionality

기본 repository 트랜잭션과 서비스 단위 작업 경계 설명. 이 프로젝트의 R01은 실제 프록시 호출 구조와 함께 판단했다.

https://docs.spring.io/spring-data/jpa/reference/jpa/transactions.html

[S5] Stripe API - Idempotent requests

동일 키의 요청 파라미터를 비교하는 실제 결제 API 계약 사례. 법적 의무 또는 모든 금융기관의 공통 규칙이라는 의미는 아니다.

https://docs.stripe.com/api/idempotent_requests

> 최종 승인 기준은 기능 존재 여부가 아니라 검증 가능한 결과입니다. 같은 요청의 중복 반영 0건, 허용 금액의 총액 보존, 타 사용자 응답 노출 0건, 민감값 로그 0건, 배치 누락 탐지·복구, 검증된 SHA의 재현 가능한 배포가 증거로 남아야 합니다.